

第 15 章 | 卷積神經網路：CNN 影像分類

學生版公開講義 | 請搭配本課程 Colab，依照段落逐步操作與自我檢查。

來源說明：課程參考《Python Machine Learning, 3rd Edition》；原始範例程式碼採 MIT License。本檔由恩恩統計家教整理為學生版學習摘要，不含原書 PDF 或掃描內容。

1. 本章學習路線

在掌握了張量運算與基礎神經網路（MLP）後，本章比較傳統全連接網路與卷積神經網路，並說明 CNN 如何保留影像的空間結構：卷積神經網路 (Convolutional Neural Networks, CNN)。

處理影像資料時，傳統 MLP 會將影像「壓扁」為一維向量，這不僅破壞了像素間的空間結構資訊，也會讓參數量快速增加。CNN 則透過模擬生物視覺系統，能夠在保留空間特徵的同時，明顯降低模型複雜度。本章將從生物學直覺出發，協助你深入數學細節，最後建立一個可重複訓練與評估的影像分類模型。

學習目標

生物起源與架構：理解 CNN 受 1959 年 Hubel 與 Wiesel 研究啟發的背景，以及特徵階層 (Feature Hierarchy) 的概念。

卷積核心要素：熟練掌握填補 (Padding)、步幅 (Stride) 與核心大小 (Kernel Size) 對特徵圖維度的影響。

池化與魯棒性：辨析最大池化 (Max-pooling) 與平均池化 (Mean-pooling) 在特徵降維與平移不變性中的角色。

實戰建立模型：學習使用 TensorFlow/Keras 建立包含權重共享、Dropout 與數值穩定損失函數的深度網路。

2. 核心觀念：CNN 為什麼像人類的大腦？

CNN 的設計並非純數學推導，而是深受 David H. Hubel 與 Torsten Wiesel (1959) 針對貓視覺皮層研究的啟發。他們發現大腦神經元具有層次化特徵：初級層級偵測簡單的邊緣與直線，高階層級則組合這些訊息以辨識複雜的形狀（如臉孔、建築）。

這種「由局部到整體」的階層化特徵提取，讓 CNN 具備了控制參數數量的兩大關鍵特性：

稀疏連接 (Sparse Connectivity)：在全連接層中，每個輸出單元都與輸入的所有單元連接。但在卷積層中，每個輸出單元僅與輸入影像的一個局部區域連接，這稱為局部感受野 (Local Receptive Field)。這模仿了生物視覺中神經元只對特定區域產生反應的特性。

參數共享 (Parameter-sharing)：CNN 假設「在影像某處有用的特徵（如垂直邊緣），在其他地方也同樣有用」。因此，同一個濾波器 (Kernel) 會掃描整張影像，共用同一組權重。

機制說明：透過這兩點，模型能自動學習特徵階層。若使用全連接層處理一個 $n_1 \times n_2$ 的影像，參數會以 $(n_1 \times n_2)^2$ 的規模快速增加；而 CNN 則將複雜度降至與核心大小相關，這正是它處理影像具有計算效率優勢的原因。

3. 卷積運算的實作細節：濾波器與特徵圖

離散卷積 (Discrete Convolution) 是將原始像素轉換為具有物理意義之特徵圖 (Feature Map) 的過程。

技術筆記 (Technical Note)：雖然學界通稱為「卷積」，但包含 TensorFlow 在內的大多數深度學習框架，實務上實作的是互相關 (Cross-correlation)。兩者差異在於互相關不需要預先對濾波器進行 180 度翻轉 (Rotation)，這在運算上更為高效。

卷積三大超參數

填補 (Padding)：

Valid： $p=0$ ，無填補。影像邊緣資訊利用率低，維度逐層縮小。

Same：自動計算 p ，使輸出維度與輸入相同，是維持空間解析度的首選。

Full： $p=m-1$ 。用於信號處理以極小化「邊界效應 (Boundary Effects)」，在 CNN 中極罕見。

步幅 (Stride, s)：控制核心移動的間隔。較大的步幅會直接導致特徵圖的子採樣。

核心大小 (Kernel Size, m)：現代趨勢如 $1 \times 1, 3 \times 3, 5 \times 5$ 。較小核心的堆疊通常比單一大型核心更有效。

空間維度計算公式

輸出維度 o 計算如下 ($\text{floor}(\cdot)$ 代表下取整符號 Floor)： $o = \text{floor}(sn + 2p - m) + 1$

範例演示：輸入 $n=10$ ，核心 $m=5$ ，Padding $p=2$ ，Stride $s=1$ 。 $o = \text{floor}(10 + 2(2) - 5) + 1 = 10$ 。此即典型的 Same Padding。

4. 子採樣與穩定性：池化層 (Pooling) 的作用

池化層透過子採樣 (Subsampling) 強化模型對雜訊與微小位移的魯棒性 (Robustness)。

特性	最大池化 (Max-pooling)	平均池化 (Mean-pooling)
運作機制	提取局部區域的最大值	計算局部區域的平均值
主要優勢	提供局部不變性 (Local Invariance)	提供特徵平滑效果
實際意義	無可學習參數。顯著減少運算量並預防過擬合。	

現代設計趨勢：雖然池化是經典 CNN 的核心，但近年部分研究（如 All Convolutional Net）傾向捨棄池化層，轉而使用 Stride=2 的卷積層來達成特徵降維，這可以看作是具備「可學習權重」的動態池化。

5. 多通道資料與張量處理 (Rank-3 & Rank-4 Tensors)

影像通常以 RGB 三通道輸入 (Rank-3 張量： $H \times W \times C_{in}$)。

卷積核的演化：當輸入通道為 C_{in} ，欲輸出 C_{out} 個特徵圖時，核心會成為一個 4D 張量 ($m_1 \times m_2 \times C_{in} \times C_{out}$)。

計算邏輯：針對每個通道進行卷積後加總，再疊加一個偏置值 (Bias) 並通過激活函數。

參數計算比較

假設輸入 $n_1 \times n_2 \times 3$ ，輸出 $C_{out}=5$ ，卷積核 $m_1 \times m_2$ ：

CNN 參數： $(m_1 \times m_2 \times 3 \times 5) + 5$

MLP 參數： $(n_1 \times n_2 \times 3) \times (n_1 \times n_2 \times 5) + (n_1 \times n_2 \times 5)$ 這再次證明了 CNN 在處理大型影像時，如何利用「局部性」控制參數數量。

6. 模型正則化與最佳化策略

Dropout 與 Inverse Dropout

為了防止過擬合，Dropout 在訓練時隨機「捨棄」神經元。TensorFlow 採用 Inverse Dropout 技術：

機制：訓練時將保留的神經元激活值除以 $(1-p_{drop})$ 。

例子：若丟棄率為 0.5，則訓練時將剩餘激活值放大 2 倍 $(1/0.5)$ 。

優點：這樣在預測階段（Inference）不需任何改動即可直接使用全部神經元，確保尺度一致。

損失函數選擇指南

在 Keras 中，請根據標籤格式進行匹配：

損失函數	適用場景	標籤格式
BinaryCrossentropy	二元分類	[0] 或 [1]
CategoricalCrossentropy	多類別	One-hot, 如 [0, 0, 1]
SparseCategoricalCrossentropy	多類別	整數標籤, 如 2

參數設定：若輸出層未套用 Sigmoid 或 Softmax，請設定 `from_logits=True`；若輸出已是機率，則設定為 `False`。兩者必須與模型輸出形式一致。

7. 重要術語對照與核心程式流程

關鍵術語

Feature Map (特徵圖)

Local Receptive Field (局部感受野)

Weight Sharing (權重共享)

NHWC / NCHW：TensorFlow 預設使用 NHWC（數量, 高, 寬, 通道）；而 PyTorch 等工具預設使用 NCHW。跨平台實作時須注意維度對齊。

MNIST 實作標準流程

根據 LeCun 經典架構的實作導讀：

資料處理：將 MNIST 像素從 `[0, 255]` 歸一化至 `[0, 1]`，確保訓練穩定。

架構設計：

`Conv2D (5×5, 32 filters) → MaxPool2D (2×2, Stride=2)`

`Conv2D (5×5, 64 filters) → MaxPool2D (2×2, Stride=2)`

`Flatten → Dense (1024) → Dropout → Dense (10)`

維度演變：`28×28×1 → 14×14×32 → 7×7×64 → Flatten`

8. 實作小幫手：錯誤排除與自我檢查

常見錯誤 (Troubleshooting)

維度缺失：影像讀入若為 (28,28)，必須使用 `tf.expand_dims` 增加通道維度變為 (28,28,1)。

損失函數標籤不匹配：標籤為 `int` 卻使用 `CategoricalCrossentropy`（應使用 `Sparse` 版本）。

模式錯誤：自定義訓練迴圈時，忘記在測試階段將 `training` 參數設為 `False`，導致 Dropout 仍在運作，結果不穩定。

自我檢查問題 (Self-Check)

情境分析：在什麼情況下你會偏好平均池化 (Average Pooling) 而非最大池化 (Max-pooling)？（提示：當你希望獲得更平滑的特徵提取，而非僅關注最顯著特徵時。）

機制理解：為何池化層不需要權重更新？（提示：它是固定數學函數，無可學習參數。）

現代架構：試解釋為什麼現代網路有時會移除池化層？（提示：利用 Strided Convolution 達成可學習的子採樣。）

穩定性：為什麼設定 `from_logits=True` 能提升計算效率？（提示：數學項抵消，避免數值溢位。）

跨框架：當你把一個 PyTorch 模型遷移到 TensorFlow 時，張量形狀最可能遇到的挑戰是什麼？（提示：NHWC 與 NCHW 的對齊。）